

Faculdade de Informática e Administração Paulista – FIAP
Ciência da Computação

**SGEC - Sistema de Gerenciamento
Energético da Colônia**

Ana Carolina Freire Mafra
Daniel Guimarães Barreto
Paulo Henrique da Silva Gola
Vinicius Daniel de Borba
Vitor de Araujo Ferreira

São Paulo, Brasil

SUMÁRIO

1 Introdução	3
2 Organização dos dados	3
3 Estruturas de controle	4
4 Regras de decisão	4/5
5 Modelo de previsão	5
6 Análise do uso da energia	6
7 Considerações	6
8 Link do repositório	7

1 Introdução

Este relatório descreve o desenvolvimento do Sistema de Gerenciamento Energético da Colônia, o *SGEC*, parte do projeto da Missão Aurora Siger. O objetivo do projeto completo é simular o gerenciamento eficaz de energia de uma colônia em Marte. Além disso, o *SGEC* tem como função principal monitorar a energia disponível, tomar decisões caso seja necessário e prever a energia solar.

Todo o sistema foi desenvolvido na linguagem Python e organizado em quatro arquivos principais: `data.py`, `energy.py`, `decison.py` e `forecast.py`, em que cada um é responsável por uma parte específica do sistema.

2 Organização dos dados

Os dados foram concentrados no arquivo `data.py`, em três estruturas de listas diferentes.

A primeira estrutura foi uma lista de dicionários (sistemas) que contém os sistemas da nave. Cada dicionário contém as chaves: nome, prioridade, consumo, status de ativação e tipo. Os sistemas cadastrados foram: Geração de Oxigênio, Habitação, Comunicações e Laboratório.

A segunda estrutura é um dicionário, tabelas hash, (`fontes_energia`) que representa as fontes de energia disponíveis. Foram definidas solar e bateria.

A terceira, e última, estrutura são duas listas (`incidencia_solar` e `energia_solar`) com dados de incidência solar e energia gerada, que são utilizadas pela regressão linear.

3 Estruturas de controle

Além dos arquivos principais do sistema, foi criado o arquivo `structure.py`, responsável por organizar os sistemas da colônia.

Nesse arquivo, os sistemas são colocados em uma fila, permitindo que sejam processados em sequência. Também foi criada uma pilha para armazenar alertas do sistema.

O módulo possui a função `buscar_maior_consumo()`, que identifica qual sistema consome mais energia, ajudando na tomada de decisões em casos de economia de energia.

Além disso, a função `ordem_prioridade()` organiza os sistemas de acordo com sua prioridade, permitindo que os sistemas mais importantes sejam mantidos ativos em situações críticas.

4 Regras de decisão

As regras de decisão foram alocadas no arquivo `decison.py`. O módulo importa as funções `calcular_energia_total()` e `calcular_consumo_total()` do arquivo `energy.py`, e isso evita a repetição de código.

A função `tomar_decisoes()` aplica três condições para determinar o estado do sistema:

Se $\text{energia} < 50$ e $\text{consumo} > 70$:

Mostrar: “ALERTA!: Ativar modo de economia.”

Senão, se $\text{consumo} > \text{energia}$:

Mostrar: “ATENÇÃO!: Reduza levemente o consumo de energia.”

Senão:

Mostrar: "Sistema energético estável."

Com os dados definidos no projeto, energia disponível de 230 unidades e consumo total de 110 unidades, o sistema retorna que a energia está suficiente e sugere armazenar o excedente.

5 Modelo de previsão

O modelo de previsão foi inserido no arquivo forecast.py, utilizando a regressão linear para fazer a estimativa da energia solar gerada com base na incidência solar. As bibliotecas utilizadas foram scikit-learn, pandas e matplotlib.

Foram importados os dados do data.py (incidência_solar e energia_solar). A variável independente X é a incidência solar e a variável dependente Y é a energia gerada. Após o treinamento, o modelo é capaz de prever valores futuros com base em uma nova entrada de incidência solar.

O coeficiente de determinação R^2 obtido foi de 0,99, indicando que o modelo representa bem a relação entre as variáveis. Ao testar com incidência solar igual a 12, a previsão retornada foi de 27,04 unidades de energia gerada. O sistema também gera um gráfico de dispersão com a reta de regressão ajustada.

6 Análise do uso de energia

A análise do uso de energia foi implementada no arquivo `energy.py` e percorre três funções.

A função `calcular_energia_total()` utiliza o dicionário `fontes_energia` e soma todos os valores correspondentes. A função `calcular_consumo_total()` utiliza a lista `sistemas` e faz a soma do consumo apenas dos que estão com `status ativo = True`. A função `analisar_energia()` usa as duas anteriores, exibe os valores e aplica as condições: Se o consumo for maior que a energia disponível, emitir alerta de consumo maior que a geração de energia; Se o excedente for maior que 50 unidades, sugerir armazenar energia excedente; Caso contrário, emitir a mensagem informando que o sistema energético está estável.

As saídas geradas foram: energia total disponível = 230; consumo total de energia = 110; sugestão: armazenar energia excedente.

7 Considerações

O sistema desenvolvido integrou os principais conceitos trabalhados ao longo da disciplina: organização de dados em estruturas como listas e dicionários, lógica de decisão com estruturas condicionais, modelagem matemática por meio de regressão linear e análise do balanço energético.

O SGEC é capaz de monitorar o consumo dos sistemas da colônia, tomar decisões automáticas para preservar os sistemas essenciais e antecipar a geração de energia solar, tornando a colônia mais eficiente e segura.

8 Link do repositório

Link do repositório no GitHub:
<https://github.com/vitorarafe/Sistemas-col-nia-Aurora-Siger.git>